



RTL LABORATORY FILE

SUBMITTED BY:

NAME	ABDUL RAFAY
------	-------------

SUBMITTED TO:

SIR KHALIL REHMAN

(RTL) WEEK # 04 LAB

Table Of Contents

VIDEO GRAPHICS CONTROLLER (VGA CONTROLLER)	-----3
AXI4-LITE MASTER AND SLAVE	-----7
AHB-LITE	-----14

RTL LAB WEEK # 04

DESIGN AND SIMULATION OF COMMUNICATION PROTOCOLS USING SYSTEMVERILOG

➤ OBJECTIVE:

To understand, design, and simulate communication protocols and display controller modules using SystemVerilog and verify their operation through RTL simulation.

LAB TASKS

- **Question 1:** In SystemVerilog design, synthesize, and simulate a Video Graphics Controller (VGA Controller) using SRAM.

❖ CODE:

• DESIGN CODE:

```
// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 01
// VIDEO GRAPHICS CONTROLLER (VGA CONTROLLER) USING SRAM
module vga_controller (
    input logic    clk,
    input logic    rst_n,
    input logic [7:0] sram_data,
    output logic [16:0] sram_addr,
    output logic    sram_oe,
    output logic [3:0] vga_r,
    output logic [3:0] vga_g,
    output logic [3:0] vga_b,
    output logic    vga_hsync,
    output logic    vga_vsync,
    output logic    vga_blank
);
    logic [9:0] h_count;
    logic [9:0] v_count;
    logic    h_visible;
    logic    v_visible;
    logic    video_active;
    localparam H_VISIBLE = 640;
    localparam H_FRONT  = 16;
    localparam H_SYNC    = 96;
    localparam H_BACK    = 48;
    localparam H_TOTAL   = 800;
    localparam V_VISIBLE = 480;
    localparam V_FRONT   = 10;
```

```

localparam V_SYNC = 2;
localparam V_BACK = 33;
localparam V_TOTAL = 525;
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        h_count <= 10'd0;
    end else if (h_count == H_TOTAL - 1) begin
        h_count <= 10'd0;
    end else begin
        h_count <= h_count + 10'd1;
    end
end
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        v_count <= 10'd0;
    end else if (h_count == H_TOTAL - 1) begin
        if (v_count == V_TOTAL - 1) begin
            v_count <= 10'd0;
        end else begin
            v_count <= v_count + 10'd1;
        end
    end
end
assign h_visible = (h_count < H_VISIBLE) ? 1'b1 : 1'b0;
assign v_visible = (v_count < V_VISIBLE) ? 1'b1 : 1'b0;
assign video_active = h_visible && v_visible;
assign vga_hsync = ((h_count >= (H_VISIBLE + H_FRONT)) &&
    (h_count < (H_VISIBLE + H_FRONT + H_SYNC))) ? 1'b0 : 1'b1;
assign vga_vsync = ((v_count >= (V_VISIBLE + V_FRONT)) &&
    (v_count < (V_VISIBLE + V_FRONT + V_SYNC))) ? 1'b0 : 1'b1;
assign vga_blank = ~video_active;
assign sram_addr = video_active ? {9'd0, v_count[8:1], h_count[9:1]} : 17'd0;
assign sram_oe = video_active;
assign vga_r = video_active ? sram_data[7:4] : 4'd0;
assign vga_g = video_active ? sram_data[3:2] : 4'd0;
assign vga_b = video_active ? sram_data[1:0] : 4'd0;
endmodule

```

• TEST BENCH CODE:

```

// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 01
// TEST-BENCH VIDEO GRAPHICS CONTROLLER (VGA CONTROLLER) USING SRAM
module tb_vga_controller;
    logic    clk;
    logic    rst_n;
    logic [7:0] sram_data;
    logic [16:0] sram_addr;
    logic    sram_oe;

```

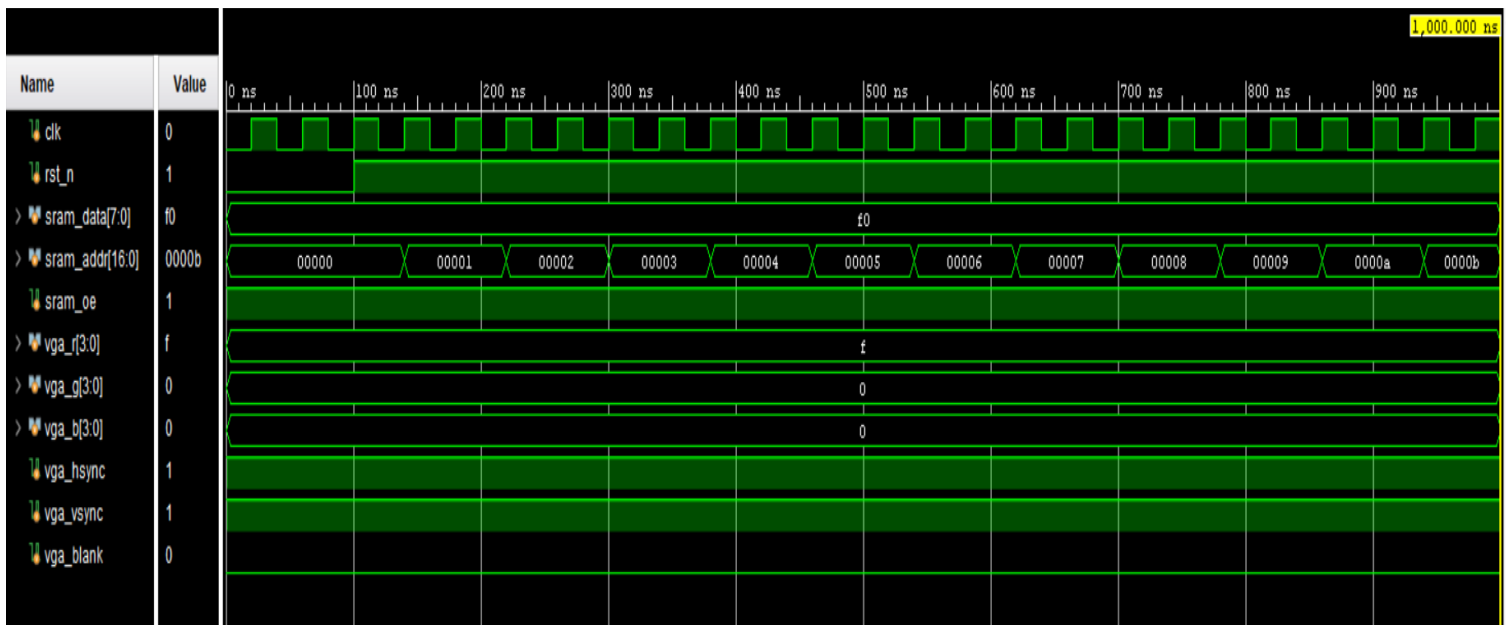
```

logic [3:0] vga_r;
logic [3:0] vga_g;
logic [3:0] vga_b;
logic      vga_hsync;
logic      vga_vsync;
logic      vga_blank;
logic [7:0] sram_mem [0:307199];
vga_controller uut (
    .clk(clk),
    .rst_n(rst_n),
    .sram_data(sram_data),
    .sram_addr(sram_addr),
    .sram_oe(sram_oe),
    .vga_r(vga_r),
    .vga_g(vga_g),
    .vga_b(vga_b),
    .vga_hsync(vga_hsync),
    .vga_vsync(vga_vsync),
    .vga_blank(vga_blank)
);
initial begin
    clk = 0;
    forever #20 clk = ~clk;
end
initial begin
    $dumpfile("vga_controller.vcd");
    $dumpvars(0, tb_vga_controller);
end
initial begin
    integer i;
    for (i = 0; i < 307200; i++) begin
        sram_mem[i] = 8'h00;
    end
    for (i = 0; i < 6400; i++) begin
        sram_mem[i] = 8'hF0;
    end
    for (i = 6400; i < 12800; i++) begin
        sram_mem[i] = 8'h0F;
    end
end
always_comb begin
    if (sram_oe) begin
        sram_data = sram_mem[sram_addr];
    end else begin
        sram_data = 8'h00;
    end
end
initial begin
    rst_n = 1'b0;
    #100 rst_n = 1'b1;
    #2000000 $finish;

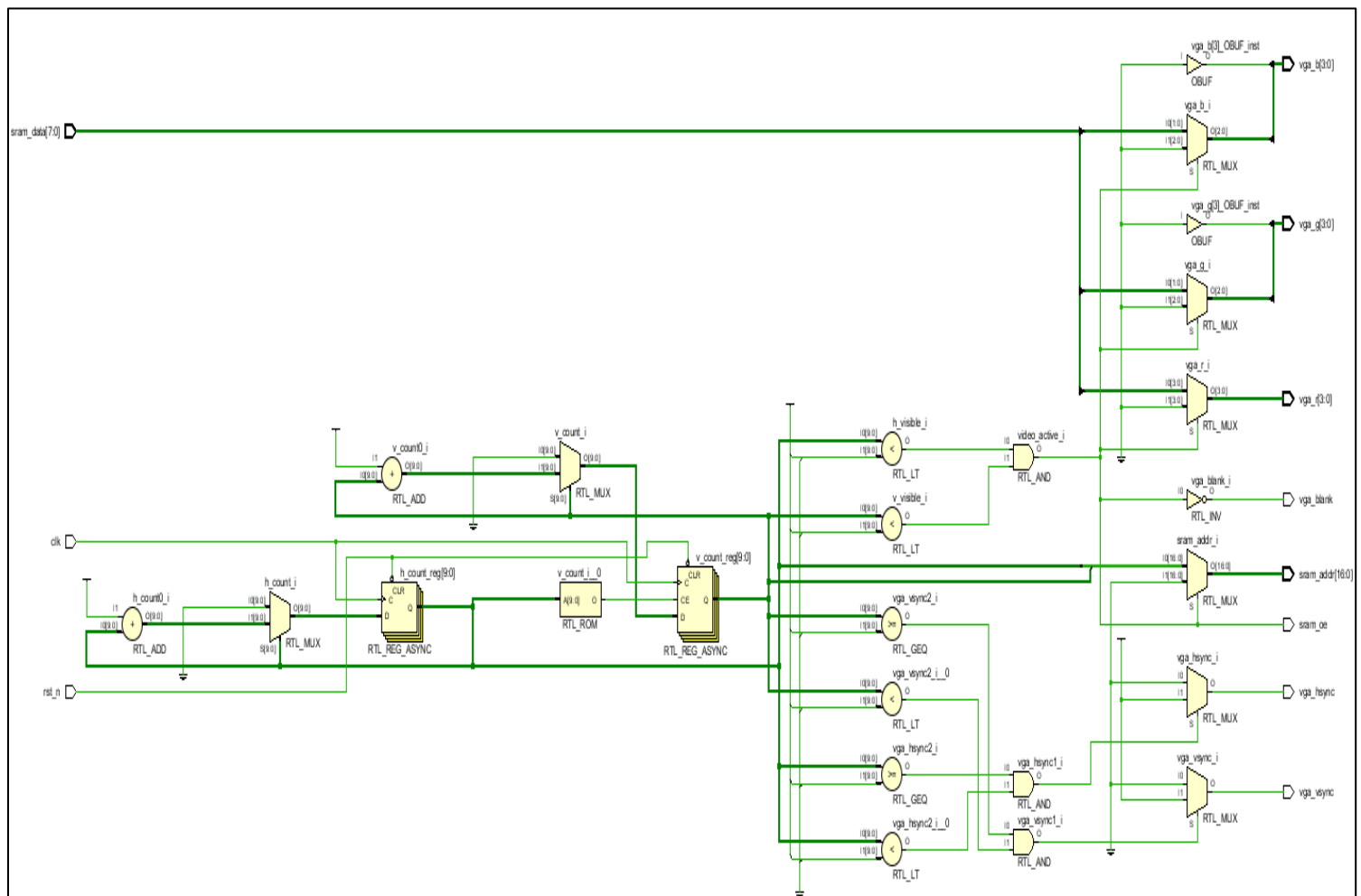
```

```
end
endmodule
```

❖ WAVE FORM OUTPUT:



❖ SCHEMATIC OUTPUT:



➤ **Question 2:** Design and simulate a AXI4-Lite Master and Slave using SystemVerilog.

❖ **CODE:**

• **DESIGN CODE:**

➤ **AXI_LITE_MASTER.SV:**

```
// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// AXI LITE MASTER
module axi_lite_master (
    input logic    clk,
    input logic    rst_n,
    input logic    start,
    input logic    wr_rd,
    input logic [7:0] addr,
    input logic [31:0] wdata,
    output logic [31:0] rdata,
    output logic    done,
    output logic    awvalid,
    input logic    awready,
    output logic [7:0] awaddr,
    output logic    wvalid,
    input logic    wready,
    output logic [31:0] wdata_out,
    output logic [3:0] wstrb,
    output logic    bready,
    input logic    bvalid,
    input logic [1:0] bresp,
    output logic    arvalid,
    input logic    arready,
    output logic [7:0] araddr,
    input logic    rvalid,
    output logic    rready,
    input logic [31:0] rdata_in,
    input logic [1:0] rresp
);
typedef enum logic [1:0] {IDLE, WRITE, READ} state_t;
state_t state, next_state;
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        state <= IDLE;
    else
        state <= next_state;
end
always_comb begin
    next_state = state;
    case (state)
        IDLE: begin
```

```

        if (start && wr_rd)
            next_state = WRITE;
        else if (start && !wr_rd)
            next_state = READ;
        end
    WRITE: if (bvalid && bready) next_state = IDLE;
    READ: if (rvalid && rready) next_state = IDLE;
endcase
end
always_comb begin
    awvalid = 1'b0;
    awaddr = 8'd0;
    wvalid = 1'b0;
    wdata_out = 32'd0;
    wstrb = 4'hF;
    bready = 1'b0;
    arvalid = 1'b0;
    araddr = 8'd0;
    rready = 1'b0;
    rdata = 32'd0;
    done = 1'b0;
    case (state)
        IDLE: begin
            awaddr = addr;
            araddr = addr;
            wdata_out = wdata;
        end
        WRITE: begin
            awvalid = 1'b1;
            awaddr = addr;
            if (awready) begin
                wvalid = 1'b1;
                wdata_out = wdata;
                if (wready) begin
                    bready = 1'b1;
                    if (bvalid && bready) begin
                        done = 1'b1;
                    end
                end
            end
        end
        READ: begin
            arvalid = 1'b1;
            araddr = addr;
            if (arready) begin
                rready = 1'b1;
                if (rvalid && rready) begin
                    rdata = rdata_in;
                    done = 1'b1;
                end
            end
        end
    end
end

```



```

    end
  endcase
end
endmodule

```

➤ AXI_LITE_SLAVE.SV:

```

// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// AXI LITE SLAVE
module axi_lite_slave (
    input logic    clk,
    input logic    rst_n,
    input logic    awvalid,
    output logic    awready,
    input logic [7:0] awaddr,
    input logic    wvalid,
    output logic    wready,
    input logic [31:0] wdata,
    input logic [3:0] wstrb,
    input logic    bready,
    output logic    bvalid,
    output logic [1:0] bresp,
    input logic    arvalid,
    output logic    arready,
    input logic [7:0] araddr,
    output logic    rvalid,
    input logic    rready,
    output logic [31:0] rdata,
    output logic [1:0] rresp
);
    logic [31:0] mem [0:255];
    typedef enum logic [1:0] {IDLE, WR_RESP, RD_RESP} state_t;
    state_t state, next_state;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= IDLE;
        else
            state <= next_state;
    end
    always_comb begin
        next_state = state;
        case (state)
            IDLE: begin
                if (awvalid && wvalid)
                    next_state = WR_RESP;
                else if (arvalid)
                    next_state = RD_RESP;
            end
        end
    end

```

```

        WR_RESP: if (bready) next_state = IDLE;
        RD_RESP: if (rready) next_state = IDLE;
    endcase
end
always_comb begin
    awready = 1'b0;
    wready = 1'b0;
    bvalid = 1'b0;
    bresp = 2'b00;
    arready = 1'b0;
    rvalid = 1'b0;
    rdata = 32'd0;
    rresp = 2'b00;
    case (state)
        IDLE: begin
            awready = 1'b1;
            arready = 1'b1;
        end
        WR_RESP: begin
            if (awvalid && wvalid) begin
                mem[awaddr] = wdata;
                bvalid = 1'b1;
                bresp = 2'b00;
            end
        end
        RD_RESP: begin
            if (arvalid) begin
                rdata = mem[araddr];
                rvalid = 1'b1;
                rresp = 2'b00;
            end
        end
    endcase
end
endmodule

```

➤ TOP_AXI_LITE.SV:

```

// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// TOP AXI LITE
module top_axi_lite (
    input logic    clk,
    input logic    rst_n,
    input logic    start,
    input logic    wr_rd,
    input logic [7:0] addr,
    input logic [31:0] wdata,
    output logic [31:0] rdata,

```

```

output logic    done
);
logic    awvalid;
logic    awready;
logic [7:0] awaddr;
logic    wvalid;
logic    wready;
logic [31:0] wdata_out;
logic [3:0] wstrb;
logic    bready;
logic    bvalid;
logic [1:0] bresp;
logic    arvalid;
logic    arready;
logic [7:0] araddr;
logic    rvalid;
logic    rready;
logic [31:0] rdata_in;
logic [1:0] rresp;
axi_lite_master master (
    .clk(clk),
    .rst_n(rst_n),
    .start(start),
    .wr_rd(wr_rd),
    .addr(addr),
    .wdata(wdata),
    .rdata(rdata),
    .done(done),
    .awvalid(awvalid),
    .awready(awready),
    .awaddr(awaddr),
    .wvalid(wvalid),
    .wready(wready),
    .wdata_out(wdata_out),
    .wstrb(wstrb),
    .bready(bready),
    .bvalid(bvalid),
    .bresp(bresp),
    .arvalid(arvalid),
    .arready(arready),
    .araddr(araddr),
    .rvalid(rvalid),
    .rready(rready),
    .rdata_in(rdata_in),
    .rresp(rresp)
);
axi_lite_slave slave (
    .clk(clk),
    .rst_n(rst_n),
    .awvalid(awvalid),
    .awready(awready),

```

```

        .awaddr(awaddr),
        .wvalid(wvalid),
        .wready(wready),
        .wdata(wdata_out),
        .wstrb(wstrb),
        .bready(bready),
        .bvalid(bvalid),
        .bresp(bresp),
        .arvalid(arvalid),
        .arready(arready),
        .araddr(araddr),
        .rvalid(rvalid),
        .rready(rready),
        .rdata(rdata_in),
        .rresp(rresp)
    );
endmodule

```

• TEST BENCH CODE:

```

// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// TEST-BENCH FOR AXI LITE
module tb_top_axi_lite;
    logic    clk;
    logic    rst_n;
    logic    start;
    logic    wr_rd;
    logic [7:0]  addr;
    logic [31:0] wdata;
    logic [31:0] rdata;
    logic    done;
    top_axi_lite dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .wr_rd(wr_rd),
        .addr(addr),
        .wdata(wdata),
        .rdata(rdata),
        .done(done)
    );
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        $dumpfile("axi_lite.vcd");
        $dumpvars(0, tb_top_axi_lite);
    end
endmodule

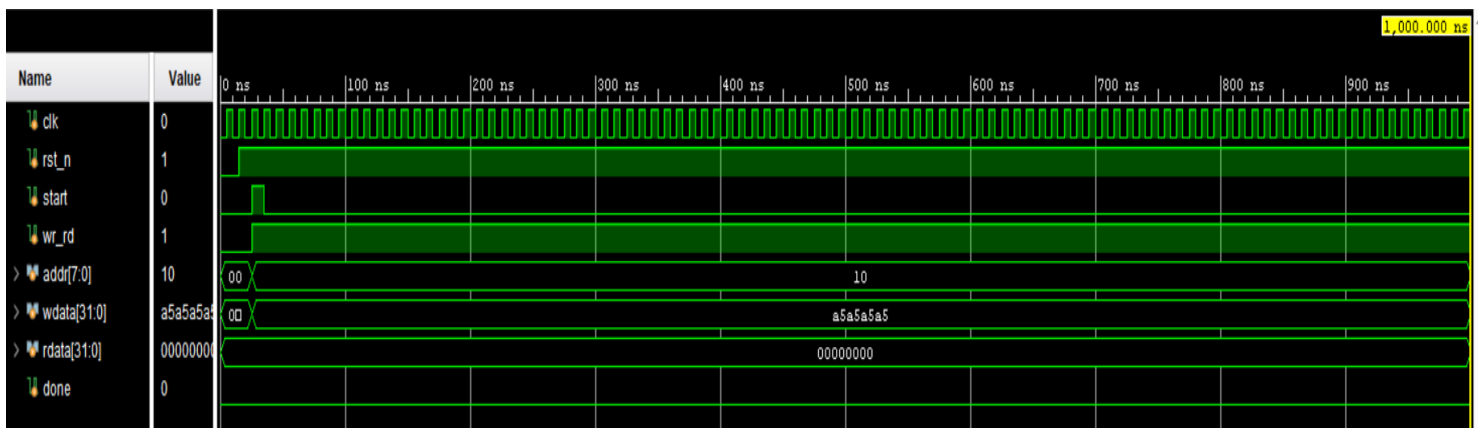
```

```

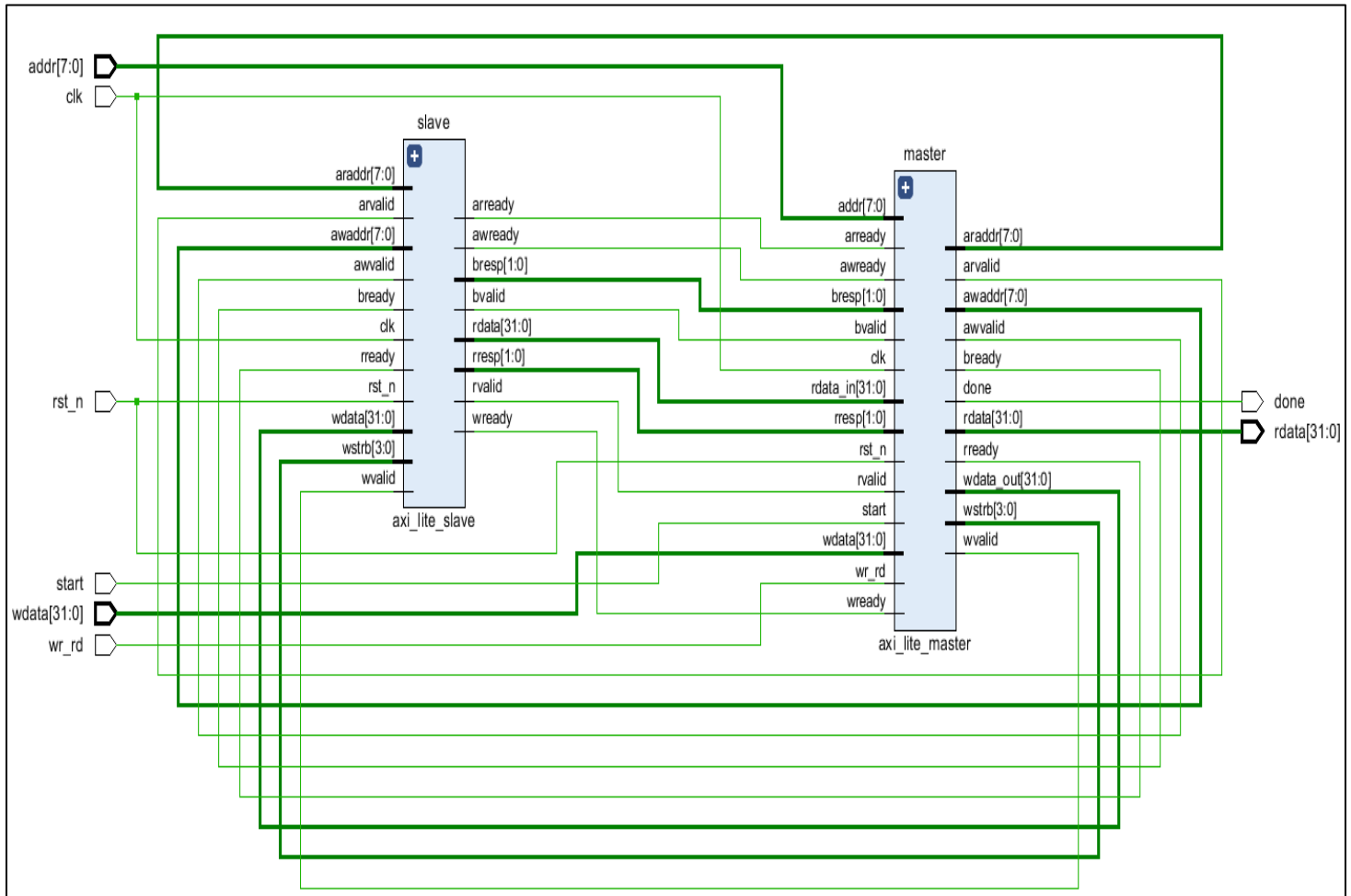
end
initial begin
    rst_n = 1'b0;
    start = 1'b0;
    wr_rd = 1'b0;
    addr = 8'd0;
    wdata = 32'd0;
    #15 rst_n = 1'b1;
    #10;
    wr_rd = 1'b1;
    addr = 8'h10;
    wdata = 32'hA5A5A5A5;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #10;
    wr_rd = 1'b1;
    addr = 8'h20;
    wdata = 32'h12345678;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #10;
    wr_rd = 1'b0;
    addr = 8'h10;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #10;
    wr_rd = 1'b0;
    addr = 8'h20;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #20;
    $finish;
end
endmodule

```

❖ WAVE FORM OUTPUT:



❖ SCHEMATIC OUTPUT:



➤ **Question 3:** Design and simulate a AHB-Lite using SystemVerilog.

❖ CODE:

• DESIGN CODE:

```
// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 02 (RTL)
// AHB SLAVE
module ahb_slave #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 32,
    parameter MEM_DEPTH = 8
)(
    input logic          HCLK,
    input logic          HRESETn,
    input logic          HSEL,
    input logic [ADDR_WIDTH-1:0] HADDR,
    input logic          HWRITE,
    input logic [1:0]     HTRANS,
```

```

input logic [2:0]      HSIZE,
input logic [DATA_WIDTH-1:0] HWDATA,
input logic      HREADY,
output logic [DATA_WIDTH-1:0] HRDATA,
output logic      HREADYOUT,
output logic      HRESP
);
localparam TRN_IDLE  = 2'b00;
localparam TRN_NONSEQ = 2'b10;
logic [DATA_WIDTH-1:0] mem [0:MEM_DEPTH-1];
logic [ADDR_WIDTH-1:0] addr_reg;
logic      write_reg;
logic      active_reg;
always_ff @(posedge HCLK or negedge HRESETn) begin
    if (!HRESETn) begin
        addr_reg <= '0;
        write_reg <= '0;
        active_reg <= '0;
    end else if (HREADY) begin
        if (HSEL && (HTRANS == TRN_NONSEQ)) begin
            addr_reg <= HADDR;
            write_reg <= HWRITE;
            active_reg <= 1'b1;
        end else begin
            active_reg <= 1'b0;
        end
    end
end
always_comb begin
    HREADYOUT = 1'b1;
    HRESP     = 1'b0;
end
always_ff @(posedge HCLK or negedge HRESETn) begin
    if (!HRESETn) begin
        HRDATA <= '0;
        for (int i = 0; i < MEM_DEPTH; i++) begin
            mem[i] <= '0;
        end
    end else begin
        if (active_reg && HREADYOUT) begin
            if (write_reg) begin
                mem[addr_reg % MEM_DEPTH] <= HWDATA;
            end else begin
                HRDATA <= mem[addr_reg % MEM_DEPTH];
            end
        end
    end
end
endmodule

```

- **TEST BENCH CODE:**

```
// // ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 02 (RTL)
// TEST BENCH FOR AHB SLAVE
`timescale 1ns / 1ps
module tb_ahb_slave;
    localparam ADDR_WIDTH = 32;
    localparam DATA_WIDTH = 32;
    localparam CLK_PERIOD = 10;
    logic          HCLK;
    logic          HRESETn;
    logic          HSEL;
    logic [ADDR_WIDTH-1:0] HADDR;
    logic          HWRITE;
    logic [1:0]      HTRANS;
    logic [2:0]      HSIZE;
    logic [DATA_WIDTH-1:0] HWDATA;
    logic          HREADY;
    logic [DATA_WIDTH-1:0] HRDATA;
    logic          HREADYOUT;
    logic          HRESP;
    ahb_slave #(
        .ADDR_WIDTH(ADDR_WIDTH),
        .DATA_WIDTH(DATA_WIDTH),
        .MEM_DEPTH(8)
    ) dut (
        .HCLK(HCLK),
        .HRESETn(HRESETn),
        .HSEL(HSEL),
        .HADDR(HADDR),
        .HWRITE(HWRITE),
        .HTRANS(HTRANS),
        .HSIZE(HSIZE),
        .HWDATA(HWDATA),
        .HREADY(HREADY),
        .HRDATA(HRDATA),
        .HREADYOUT(HREADYOUT),
        .HRESP(HRESP)
    );
    always #(CLK_PERIOD / 2) HCLK = ~HCLK;
    task ahb_write(input logic [ADDR_WIDTH-1:0] addr, input logic [DATA_WIDTH-1:0] data);
        @(posedge HCLK);
        HSEL <= 1'b1;
        HADDR <= addr;
        HWRITE <= 1'b1;
        HTRANS <= 2'b10;
        @(posedge HCLK);
        HTRANS <= 2'b00;
```

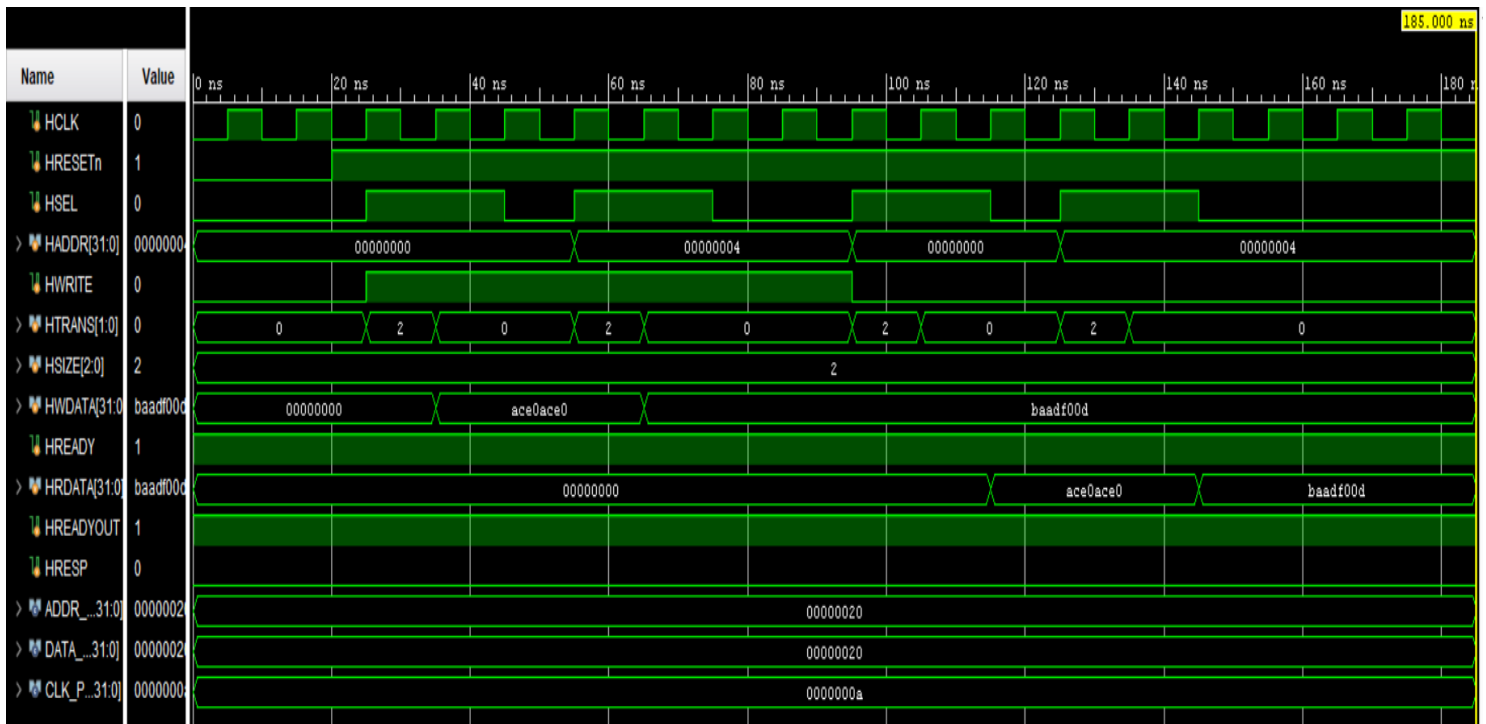


```

    HWDATA <= data;
    @(posedge HCLK);
    HSEL <= 1'b0;
endtask
task ahb_read(input logic [ADDR_WIDTH-1:0] addr);
    @(posedge HCLK);
    HSEL <= 1'b1;
    HADDR <= addr;
    HWRITE <= 1'b0;
    HTRANS <= 2'b10;
    @(posedge HCLK);
    HTRANS <= 2'b00;
    @(posedge HCLK);
    HSEL <= 1'b0;
endtask
initial begin
    HCLK = 0;
    HRESETn = 0;
    HSEL = 0;
    HADDR = 0;
    HWRITE = 0;
    HTRANS = 2'b00;
    HSIZE = 3'b010;
    HWDATA = 0;
    HREADY = 1;
    #20;
    HRESETn = 1;
    $display("[TB] Reset Deasserted. Starting AHB Transfers...");
    ahb_write(32'h0000_0000, 32'hACE0_ACE0);
    ahb_write(32'h0000_0004, 32'hBAAD_F00D);
    #20;
    ahb_read(32'h0000_0000);
    ahb_read(32'h0000_0004);
    #40;
    $display("[TB] AHB Simulation Completed Successfully!");
    $finish;
end
endmodule

```

❖ WAVE FORM OUTPUT:



❖ SCHEMATIC OUTPUT:

